SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Online Quiz Management System

Project Type: Software Engineering Project

---

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to describe the functional and non-functional requirements of the Online Quiz Management System. It defines the features, interfaces, constraints, and expected behaviour of the system.

This document serves as a reference for developers, testers, project team members, and evaluators throughout the software development process.

1.2 Scope

The Online Quiz Management System is a web-based application that allows users to view available quizzes, access quiz details, attempt quizzes, and view their results. It also provides administrative functionality for managing quizzes and questions.

The system aims to make quiz creation, participation, evaluation, and result management easier and more efficient.

The application consists of a React-based frontend, a Node.js and Express backend, and a MongoDB database.

1.3 Objectives

- To provide a simple and user-friendly online quiz platform.
- To display available quizzes with relevant information.
- To allow users to attempt quizzes online.
- To calculate quiz scores automatically.
- To allow administrators to manage quizzes and questions.
- To store quiz information and results systematically.
- To reduce manual effort in conducting and evaluating quizzes.

1.4 Definitions and Abbreviations

Term	Description
SRS	Software Requirements Specification
User	A person who uses the quiz application
Administrator	A person responsible for managing quizzes and questions
Quiz	A collection of questions on a particular subject
API	Application Programming Interface
UI	User Interface
CRUD	Create, Read, Update, Delete
DBMS	Database Management System

2. Overall Description

2.1 Product Perspective

The Online Quiz Management System is a web application that follows a client-server architecture.

The frontend provides the user interface, the backend processes requests and applies application logic, and the database stores information related to quizzes, questions, users, and results.

2.2 Product Functions

The major functions of the system are:

1. Displaying available quizzes.
2. Viewing quiz descriptions and details.
3. Searching and sorting quizzes.
4. Filtering quizzes by category.
5. Registering and logging in users, where authentication is enabled.
6. Attempting quizzes and submitting answers.
7. Automatically evaluating submitted answers.
8. Displaying quiz results and previous attempts.
9. Allowing administrators to create, update, view, and delete quizzes and questions.

2.3 User Classes

1. Student/User

- Views the list of available quizzes.
- Searches for quizzes by title or course.
- Views quiz details.
- Attempts quizzes and submits answers.
- Views scores and previous attempts, where supported.

2. Administrator

- Creates and manages quizzes.
- Adds, updates, and deletes questions.
- Sets quiz duration, category, and availability.
- Reviews quiz information and results, where supported.

2.4 Operating Environment

- Operating System: Windows, Linux, or macOS.
- Frontend: React.js, HTML, CSS, and JavaScript.
- Backend: Node.js and Express.js.
- Database: MongoDB.
- Browser: Google Chrome, Microsoft Edge, Mozilla Firefox, or another modern browser.
- Development Tools: Visual Studio Code and Git/GitHub.

2.5 Design and Implementation Constraints

- The application requires a compatible web browser.
- The backend must be available for frontend API requests.
- Database operations depend on a valid MongoDB connection.
- The application must validate user inputs before processing them.
- Administrative functions must be protected from unauthorized access.
- The system must follow the agreed project structure and API specifications.

2.6 Assumptions and Dependencies

- Users have access to a device with an internet connection or access to the locally hosted application.
- The backend and database are configured correctly.
- Quiz data is available in the database or through approved sample data.
- Users provide valid information when required.
- Authentication and role-based access are required if user accounts and administrative access are enabled.

3. Functional Requirements

3.1 Requirements Table

ID	Functional Requirement	Description	Priority
FR-01	View Available Quizzes	The system shall display quizzes that are available to users.	High
FR-02	View Quiz Details	The system shall display a quiz title, description, category, course code, and other available details.	High
FR-03	Search Quizzes	The system shall allow users to search for quizzes using relevant text such as title or course code.	Medium
FR-04	Sort Quizzes	The system shall allow quiz listings to be sorted by supported fields such as title, duration, or creation date.	Medium
FR-05	Filter Quizzes	The system shall allow users to filter quizzes by category when categories are available.	Medium
FR-06	User Registration	The system shall allow new users to create accounts if registration is enabled.	Medium
FR-07	User Login	The system shall authenticate registered users before providing access to protected features.	High
FR-08	Attempt Quiz	The system shall allow an authorized user to answer questions in a selected quiz.	High
FR-09	Submit Answers	The system shall accept and process answers submitted by the user.	High

FR-10	Automatic Evaluation	The system shall calculate scores based on submitted answers and the configured marking scheme.	High
FR-11	Display Results	The system shall display the user's score and other supported result information.	High
FR-12	Manage Quizzes	The administrator shall be able to create, view, update, and delete quizzes.	High
FR-13	Manage Questions	The administrator shall be able to add, update, and remove questions associated with quizzes.	High
FR-14	Configure Quiz	The administrator shall be able to configure quiz duration, category, and availability.	Medium
FR-15	View Previous Attempts	The system shall allow users to review their previous quiz attempts when this feature is enabled.	Medium
FR-16	Handle Errors	The system shall display appropriate messages when a request fails or no quizzes are available.	High
FR-17	Validate Inputs	The system shall validate submitted data and reject invalid inputs.	High
FR-18	Protect Administrative Access	The system shall restrict administrative operations to authorized administrators.	High

Note: Requirements that are not yet implemented represent the intended system specification and should be verified against the final application.

4. Non-Functional Requirements

ID	Requirement	Description
NFR-01	Performance	The system should display quiz listings and process normal requests within an acceptable response time.
NFR-02	Usability	The interface should be simple, consistent, and easy to understand.
NFR-03	Security	The system must protect credentials and prevent unauthorized access to protected operations.

NFR-04	Reliability	The system should handle errors without crashing and provide useful feedback.
NFR-05	Availability	The application should be accessible whenever the required server and database services are running.
NFR-06	Maintainability	The code should be organized into reusable frontend components, backend routes, and supporting modules.
NFR-07	Compatibility	The interface should work with commonly used modern web browsers.
NFR-08	Data Integrity	Quiz information, questions, and results must be stored and processed consistently.
NFR-09	Scalability	The application structure should support adding more quizzes, questions, and users.
NFR-10	Accessibility	Text, navigation, and controls should be readable and usable by a wide range of users.

5. External Interface Requirements

5.1 User Interface

The application should provide:

- A home page or dashboard.
- An Available Quizzes page.
- Quiz detail and attempt pages, when implemented.
- Login and registration pages, when authentication is enabled.
- Result and previous-attempt pages, when supported.
- Administrative pages for managing quizzes and questions.

The interface should provide clear navigation, readable text, appropriate buttons, and feedback for loading, empty, and error states.

5.2 Software Interfaces

Frontend: React.js communicates with the backend through HTTP API requests.

Backend: Node.js and Express.js handle incoming requests, validate data, apply application logic, and return responses.

Database: MongoDB stores application data such as quizzes, questions, users, and quiz attempts, depending on the implemented schema.

Version Control: Git and GitHub are used to maintain source code and track development changes.

5.3 Communication Interfaces

- The frontend and backend communicate through HTTP or HTTPS.
- Data exchanged through the API should use JSON where applicable.
- API responses should include suitable status codes and error information.
- Production deployments should use HTTPS to protect data transmitted over the network.

6. Data Requirements

The system may maintain the following logical data entities.

6.1 User

Attribute	Description
User ID	Unique identifier for a user
Name	User's display name
Email	User's email address
Password Hash	Securely stored password hash
Role	User or administrator

6.2 Quiz

Attribute	Description
Quiz ID	Unique identifier for a quiz
Title	Name of the quiz
Description	Brief explanation of the quiz
Category	Subject or topic category
Course Code	Associated course identifier
Total Questions	Number of questions in the quiz

Duration	Time allowed to complete the quiz
Availability	Indicates whether the quiz is available
Created At	Date and time of creation

6.3 Question

Attribute	**Description**
Question ID	Unique identifier for a question
Quiz ID	Identifier of the associated quiz
Question Text	The question displayed to the user
Options	Available answer choices
Correct Answer	Answer used for evaluation
Marks	Score assigned to the question

6.4 Quiz Attempt and Result

Attribute	**Description**
Attempt ID	Unique identifier for an attempt
User ID	Identifier of the participating user
Quiz ID	Identifier of the attempted quiz
Submitted Answers	Answers submitted by the user
Score	Marks obtained
Start Time	Time the attempt began
Submission Time	Time the attempt was submitted
Status	Attempt status, such as completed

These tables describe logical data requirements. In MongoDB, the corresponding information may be represented using documents and collections rather than relational tables.

7. Use Case Descriptions

7.1 Use Case: View Available Quizzes

Actor: Student/User

Precondition: The application is accessible.

Main Flow:

1. The user opens the application.
2. The user navigates to the Available Quizzes page.
3. The frontend requests available quizzes from the backend.
4. The backend retrieves the relevant quiz information.
5. The application displays the quiz list.
6. The user selects a quiz to view its details.

Alternative Flow:

- If no quizzes are available, the system displays an appropriate empty-state message.
- If the request fails, the system displays an error message.

Postcondition: The user can view the available quizzes and select one.

7.2 Use Case: Attempt a Quiz

Actor: Student/User

Precondition: A quiz is available, and the user satisfies any access requirements.

Main Flow:

1. The user selects a quiz.
2. The system displays the quiz instructions and questions.
3. The user selects answers.
4. The user submits the quiz.
5. The backend validates the submission.
6. The system evaluates the answers.
7. The system records the attempt and displays the result.

Alternative Flow:

- Invalid submissions are rejected with a suitable message.
- If the quiz expires, the system follows the configured time-limit policy.

Postcondition: The submitted attempt is evaluated and its result is available.

7.3 Use Case: Manage Quizzes

Actor: Administrator

Precondition: The administrator is authenticated and authorized.

Main Flow:

1. The administrator opens the quiz management section.
2. The administrator selects an operation: create, view, update, or delete.
3. The administrator enters or modifies quiz information.
4. The system validates the information.
5. The backend processes the request.
6. The system confirms success or displays an error.

Postcondition: The quiz information is updated when the operation succeeds.

8. System Constraints and Validation

- Required fields must not be left empty.
- Invalid or malformed requests must be rejected.
- Only available quizzes should appear in the normal available-quiz listing.
- Quiz submissions must be evaluated using the configured answer key and marking scheme.
- Protected operations must verify user permissions.
- Errors caused by unavailable backend or database services must be handled gracefully.
- Passwords must not be stored as plain text.

9. Acceptance Criteria

The project will be considered acceptable when the agreed requirements are implemented and tested.

1. Available quizzes are retrieved and displayed correctly.
2. Quiz details are shown accurately.
3. Supported search, sorting, and filtering functions return the expected results.
4. Quiz attempts and submissions work correctly, if included in the final scope.

5. Automatic evaluation produces correct scores, if implemented.
6. Administrative quiz-management operations work for authorized users.
7. Invalid inputs and failed API requests produce appropriate error messages.
8. The application communicates correctly with the configured database.
9. The interface is usable in supported browsers.
10. The implemented features pass the agreed test cases.

10. Conclusion

The Online Quiz Management System provides a structured platform for managing and participating in online quizzes. This SRS defines the expected functionality, quality requirements, interfaces, data requirements, and major use cases of the application.

The document serves as a baseline for implementation, testing, maintenance, and future enhancement. Any changes to the project scope or requirements should be reviewed and documented by the project team.